

PARCTICAL-01

Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how

CODE:-

```
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ cat program-01.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    printf("Enter Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid == 0)
    {
        printf("\nChild Process");
        printf("\nChild PID : %d", getpid());
        printf("\nParent PID: %d\n", getppid());

        execlp(command, command, NULL);

        perror("Execution Failed");
        exit(1);
    }
    else if (pid > 0)
    {
        printf("\nParent Process");
        printf("\nParent PID: %d", getpid());

        wait(NULL);

        printf("\nChild process completed.\n");
    }
    else
    {
        printf("Fork Failed!\n");
    }

    return 0;
}
```

OUTPUT:-

```
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ nano program-01.c
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ gcc program-01.c -o program
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ ./program
Enter Linux command: ^[[B

Parent Process

Child Process
Child PID : 691
Parent PID: 690
Execution Failed: No such file or directory
Parent PID: 690
Child process completed.
venu@LAPTOP-KHLR30TF:~/os-lab/practical$ cat program-01
```